CLASE 29

Django URLs y Views

El GPS de tu aplicación web

Hoy aprenderán:

- Sistema URLconf en profundidad
- URLs con parámetros dinámicos
- Organizar URLs por app (include)
- Views con lógica real
- Pasar datos del URL a la vista

Objetivo:
Definir la estructura de URLs del Dashboard E-commerce

Repaso: Lo que tenemos

Estructura actual:

```
dashboard/
  dashboard/
    settings.py
    urls.py          ← Todas las URLs aquí
  core/              ← App principal
    views.py         (home, about, contact)
  manage.py
```

URLs actuales:

```
/          → home
/about/    → about
/contact/  → contact
/admin/    → admin
```

El problema:

- Todas las URLs en UN archivo
- No hay URLs dinámicas
- No está organizado por app

Hoy lo mejoramos

URLconf: El sistema de rutas de Django

URLconf = URL Configuration — La lista de patrones que Django revisa

Usuario visita: /productos/5/

↓

Django busca en urlpatterns (de arriba a abajo)

↓

Encuentra match con el patrón

↓

Ejecuta la vista correspondiente con id=5

↓

La vista devuelve una respuesta

Si no encuentra ningún match ⟶ Error 404

Django revisa en ORDEN — el primero que coincida gana

path() — Definir rutas

Sintaxis de path():

ruta
El patrón de URL (string)

vista
La función que maneja la petición

kwargs
Argumentos extra (opcional)

name
Nombre para referenciar (opcional pero útil)

Ejemplos:

```
path('', home)                  →  /
path('about/', about)           →  /about/
path('productos/', lista)       →  /productos/
path('productos/nuevo/', ...) →
/productos/nuevo/
```

Convenciones:

- Las rutas NO empiezan con /
- Las rutas SÍ terminan con /
- Usar nombres descriptivos

El name es como un apodo para la URL

URLs estáticas vs dinámicas

URL estática:

```
/about/
/contacto/
/productos/
```

Siempre igual, una vista por URL

URL dinámica:

```
/productos/1/
/productos/2/
/productos/999/
/usuarios/juan/
```

Parte variable, UNA vista maneja todas

El problema sin URLs dinámicas:

- 100 productos → 100 vistas?
- 1000 usuarios → 1000 vistas?

La solución:

Capturar el valor variable como parámetro — Eficiencia y escalabilidad

URLs con parámetros

Sintaxis para capturar valores:

```
path('productos/<int:id>/', producto_detalle)
```

tipo ↑ *↑ nombre*

<int:id> significa:

- Captura un NÚMERO ENTERO
- Lo llama "id"
- Lo pasa a la vista como argumento

La vista recibe el parámetro:

```
def producto_detalle(request, id):
    # id contiene el valor capturado
```

```
Visita /productos/5/    → id = 5
Visita /productos/123/  → id = 123
```

Implementémoslo juntos

Convertidores de tipo

Django incluye convertidores para validar y convertir automáticamente:

Convertidor	Coincide con	Ejemplo
int	Números enteros	<int:id>
str	Cualquier texto (sin /)	<str:nombre>
slug	Letras, números, -, _	<slug:titulo>
uuid	UUID formato estándar	<uuid:codigo>
path	Texto incluyendo /	<path:ruta>

```
<int:id>
/producto/5/      ✓
/producto/abc/    ✗ (404)
```

```
<slug:nombre>
/cat/electronica/      ✓
/cat/ropa-mujer/       ✓
```

Múltiples parámetros en una URL

Puedes capturar varios valores:

```
path('ventas/<int:año>/<int:mes>/', ventas_mes)
```

`/ventas/2024/03/  captura  año=2024, mes=3`

Otro ejemplo:

```
path('usuario/<str:username>/post/<int:post_id>/', ...)
/usuario/juan/post/5/  →  username="juan", post_id=5
```

La vista recibe todos los parámetros:

```
def ventas_mes(request, año, mes):
```

Los nombres de los parámetros deben coincidir entre URL y vista

EJERCICIO RÁPIDO: URLs con parámetros

Crear estas URLs y vistas:

1	`/cliente/<int:id>/`	Muestra "Perfil del cliente #X"
2	`/buscar/<str:termino>/`	Muestra "Buscando: X"
3	`/reporte/<int:año>/<int:mes>/<int:dia>/`	Muestra "Reporte del día/mes/año"
4	`/blog/<slug:titulo>/`	Muestra "Artículo: X"

Para cada una:
Crear la vista → Agregar el path → Probar en navegador

Tiempo: 8 minutos

include() — Organizar URLs por app

El problema: Todas las URLs en UN archivo = CAOS

La solución: Cada app tiene su urls.py

Estructura con include():

```
dashboard/urls.py
```

→ `include('core.urls')` → /

→ `include('productos.urls')` → /productos/

→ `include('dashboard_app.urls')` → /dashboard/

Todo lo que empiece con /productos/ → lo maneja productos/urls.py

BREAK — 10 MINUTOS

Hasta ahora:

- URLs con parámetros
- Convertidores (int, str, slug)
- include() para organizar

Volvemos con:

- Views con lógica real
- Query parameters
- Ejercicio integrador

Views con lógica real

Hasta ahora: Views que solo devuelven texto

Ahora: Views con lógica de negocio

Una vista puede:

- ✅ Consultar datos (variables, listas, base de datos)
- ✅ Procesar información
- ✅ Tomar decisiones (if/else)
- ✅ Devolver diferentes respuestas

Evolución del proyecto:

Por ahora:
Datos en la vista

→

Próxima clase:
Templates (HTML)

→

Después:
Models (BD)

Query Parameters — Datos en la URL

```
/productos/?orden=precio&direccion=asc
```

Query string (después del ?)

Acceder en la vista:

```
request.GET.get('orden', 'nombre')
```

`.get('nombre', 'default')` evita errores si no existe

Usos comunes:

- Búsqueda: ?q=laptop
- Filtros: ?categoria=tech&precio_max=500
- Ordenamiento: ?orden=precio
- Paginación: ?pagina=2

Diferencia clave:

Parámetros en la ruta
/productos/<int:id>/ → Identifica el recurso

Query parameters
?orden=precio&dir=asc → Filtros opcionales

Tipos de respuesta

HttpResponse

Respuesta HTML / texto

```
HttpResponse("<h1>Hola</h1>")
HttpResponse("Texto plano")
HttpResponse(content, status=404)
```

Uso: Páginas web (HTML)

JsonResponse

Respuesta JSON (para APIs)

```
JsonResponse({'msg': 'Hola'})
JsonResponse(lista, safe=False)
```

Uso: APIs, respuestas para JavaScript

Próxima clase: render() para templates HTML — Separa el HTML del Python. Por ahora usamos HttpResponse. Templates harán esto mucho más elegante.

PROYECTO: Estructura completa del Dashboard

APP CORE

Páginas públicas

/ → Landing page

/about/ → Sobre nosotros

/contact/ → Contacto

APP PRODUCTOS

Catálogo

/productos/ → Lista

/productos/<int:id>/ → Detalle

/productos/categoria/<str>/ → Filtro

/productos/api/ → API JSON

APP DASHBOARD

Panel (crear ahora)

/dashboard/ → Panel principal

/dashboard/ventas/ → Ventas

/dashboard/ventas/<int:año>/ → Por año

Crear app dashboard_app y definir todas las rutas con include()

Tiempo: 15 min

Revisión: Estructura de URLs completa

CORE
- Landing
- About
- Contact

PRODUCTOS
- Lista
- Detalle
- Categoría
- API

DASHBOARD
- Panel
- Ventas
- Ventas año

ADMIN
- Admin

Todas las URLs organizadas por app — Código limpio y mantenible
Cada app es independiente y se puede reutilizar en otros proyectos

Resumen — Sistema de URLs

URL	Captura
path('', vista)	URL raíz
path('pagina/', vista)	URL estática
path('<int:id>/', vista)	Número entero
path('<str:nombre>/', vista)	Cualquier texto
path('<slug:titulo>/', vista)	Texto URL-friendly

Organización	Función
include('app.urls')	Delegar a otra app
app_name = 'productos'	Namespace
name='detalle'	Nombre para la URL

Query params:

```
request.GET.get('param', 'default')
```

Resumen — Django URLs y Views

Lo que logramos hoy:

- URLs con parámetros dinámicos
- Convertidores (int, str, slug)
- Múltiples parámetros en una URL
- include() para organizar por app
- Query parameters (request.GET)
- Views con lógica real
- JsonResponse para APIs
- Estructura completa del proyecto

PRÓXIMA CLASE: Templates

- HTML en archivos separados
- Variables dinámicas {{ variable }}
- Tags: {% if %}, {% for %}
- Template inheritance (extends)
- Static files (CSS, imágenes)

Proyecto Dashboard E-commerce:

Setup base (Clase 28)
URLs definidas (Clase 29)
→ **Templates con diseño real (Clase 30)**